University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

MoveLink

Echtzeit-Bewegungsanalyse im Krafttraining

Pflichtenheft

Software-Architekturen Labor

Sommersemester 2026

Betreuer: Prof. Dr. Sinz

Team: Erlind
Luca Schöneberg
Devin Uyan

Abgabe: 08. April 2026

Version 1.0 – Stand: 3. April 2026

1 Grundlegende Problemstellung

Bei der Ausübung von Krafttraining ist die korrekte Bewegungsausführung entscheidend für den Trainingserfolg und die Verletzungsprävention. Trainierende erhalten jedoch selten unmittelbares, objektives Feedback zu ihrer Bewegungsqualität.

MoveLink adressiert dieses Problem: Trainingsgeräte (z. B. Hanteln) werden mit Sensoren ausgestattet, die Bewegungsdaten in Echtzeit erfassen und drahtlos an eine Anwendung übertragen. Dort wird dem Trainierenden visuelles Feedback gegeben.

Im Rahmen dieses Laborprojekts wird ein **Proof of Concept** entwickelt, der die gesamte Datenkette – von der Sensorerfassung über die Übertragung bis zur Echtzeit-Visualisierung – prototypisch umsetzt. Die Bewegungserkennung via Machine Learning ist Gegenstand eines separaten Projekts und wird hier als Datenquelle abstrahiert.

2 Architekturvorschlag

2.1 Gewählte Architektur: Client-Server mit BLE-Gateway

Die Architektur folgt einem **dreischichtigen Client-Server-Modell**:

Abbildung 1: Übersicht der Systemarchitektur

- **Embedded:** XIAO nRF52840 erfasst Bewegungsdaten (Accelerometer, Gyroskop) und sendet sie via **BLE**.
- **Backend:** Lokal in einem Docker-Container betriebener Server als BLE-Gateway; leitet Daten über **WebSockets** weiter und bietet eine **REST-API** für Konfiguration.
- **Frontend:** Web-Applikation zur Echtzeit-Visualisierung der Bewegungsdaten.

2.2 Diskussion von Alternativen

Alternative	Vorteile	Nachteile
Direkt BLE → App (kein Server)	Geringere Latenz; weniger Infrastruktur	Web Bluetooth API eingeschränkt; plattformabhängig; keine zentrale Verarbeitung
MQTT-Broker (IoT-Standard)	Skalierbar; Pub/Sub ideal für Sensordaten	Infrastruktur; Overhead für PoC überproportional
Native App (Flutter/MAUI)	Direkter BLE-Zugriff; native Performance	Aufwand; schwerer im Team zu parallelisieren

Tabelle 1: Vergleich der Architektur-Alternativen

Begründung: Die Client-Server-Architektur ermöglicht eine klare Trennung der Verantwortlichkeiten (Embedded / Backend / Frontend), erleichtert parallele Teamarbeit und liefert eine plattformunabhängige Web-Oberfläche. Das gesamte System wird **lokal** über Docker Compose betrieben – es ist kein externer Server oder Cloud-Hosting erforderlich.

3 Technologieauswahl

Schicht	Technologie	Begründung
Embedded	XIAO nRF52840, C/C++	Integriertes BLE; kompakt; stromsparend
Backend	Node.js oder Python	BLE-Bibliotheken (Noble/Bleak); WebSocket-Support; containerisierbar
Frontend	React (TypeScript)	Komponentenbasiert; Visualisierungs-Ökosystem (D3.js, Recharts)
Echtzeit	WebSockets	Bidirektional; geringe Latenz; nativ im Browser
Container	Docker Compose	Lokaler Betrieb aller Dienste; reproduzierbar; kein externes Hosting nötig
Datenformat	JSON	Leichtgewichtig; universell; gut für Debugging

Tabelle 2: Technologieübersicht

4 Use Cases

UC-1: Echtzeit-Training überwachen (Primär)

Akteur: Trainierender
Vorbedingung: Sensor verbunden

1. Trainierender startet eine Trainingseinheit in der App.
2. Sensor erfasst Bewegungsdaten und sendet sie via BLE an den Server.
3. Server leitet die Daten per WebSocket an das Frontend.
4. Frontend visualisiert die Bewegung in Echtzeit ($< 1\,\mathrm{s}$ Latenz).

UC-2: Sensorverbindung herstellen

Akteur: Trainierender
Vorbedingung: Sensor eingeschaltet, Server läuft

1. App zeigt verfügbare Sensoren an.
2. Trainierender wählt Sensor aus; Verbindung wird hergestellt.

UC-3: Trainingsdaten einsehen (Sekundär)

Akteur: Trainierender
Vorbedingung: Vergangene Trainingseinheit vorhanden

1. Trainierender navigiert zur Übersicht vergangener Einheiten.
2. App zeigt historische Bewegungsdaten an.

5 Muss- und Kann-Kriterien

Muss-Kriterien:

M1 Echtzeitübertragung der Sensordaten mit einer Latenz von **max. 1 s**.

M2 **Visuelle Darstellung** der Bewegungsdaten als Live-Diagramm.

M3 **Eigenständige Web-Applikation** (kein Plugin / nativer Client nötig).

M4 Plattformübergreifend nutzbar: **Android (Browser), macOS, Windows**.

M5 Klare **Trennung der Systemkomponenten** (Embedded, Backend, Frontend).

M6 **Lokaler Betrieb** des gesamten Systems via Docker Compose (kein Cloud-Hosting).

Kann-Kriterien:

K1 Einfache **Bewertung der Bewegungsausführung** (Schwellenwert-basiert).

K2 **Persistierung** von Trainingsdaten für spätere Einsicht.

K3 **3D-Visualisierung** der Sensorbewegung (z. B. Three.js).

K4 Unterstützung **mehrerer Sensoren** gleichzeitig.

K5 **Konfigurierbarkeit** der Darstellung (Achsen, Zeitfenster).